College of Basic and Applied Sciences

School of Engineering Sciences · Department of Computer Engineering

CPEN 208 — Project 2

API / Web Service implementing the Project 1 functionalities

Name: David Kwame Odoi-Anim

Student ID: 22312110

Course: CPEN 208 — Introduction to Software Engineering

Semester: Second Semester, 2025/2026

Lecturer: Mr. Asiamah

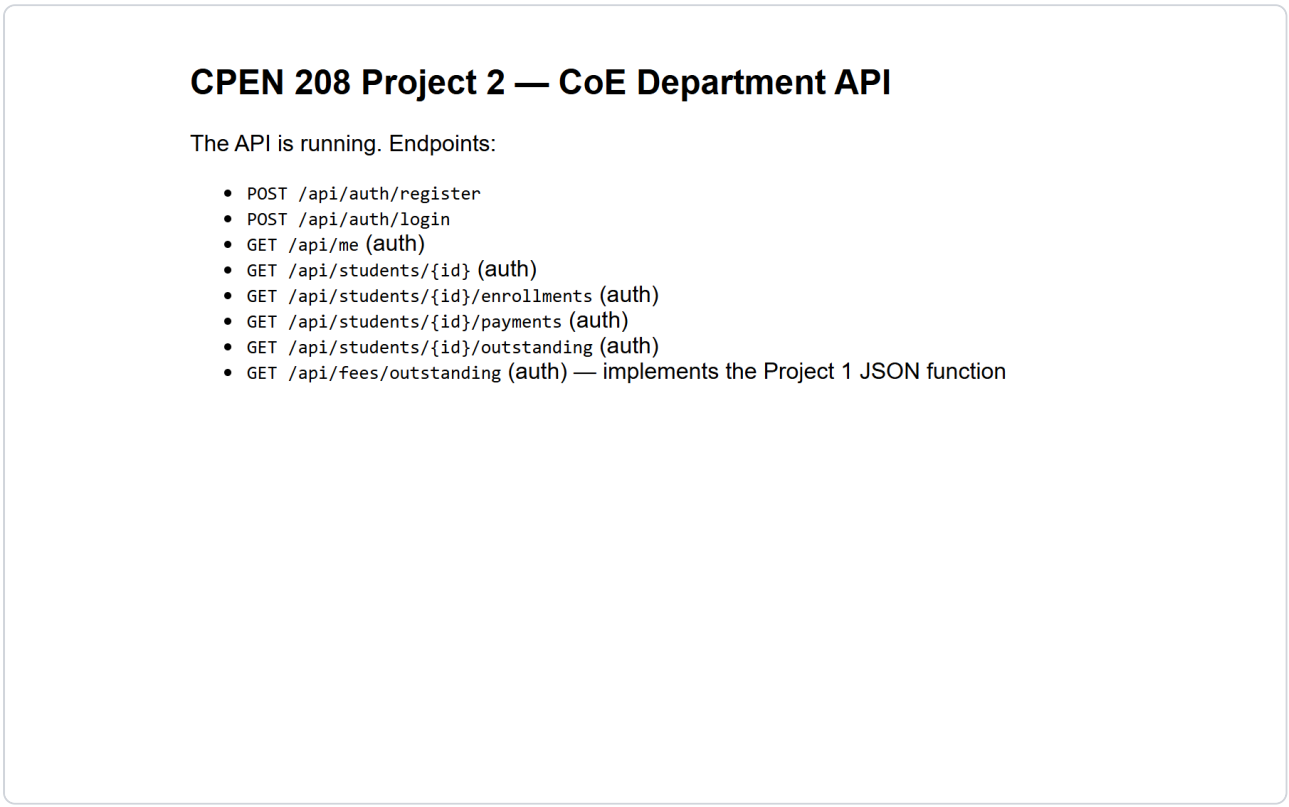
Spring Boot 3.5 · Java 21 · Spring JDBC · JWT · BCrypt

GitHub: `https://github.com/doxaodoi/project-1-2`

1. Overview

Project 2 is an **API / web service** that implements the functionalities created in Project 1 and exposes them over HTTP as JSON. It is a Spring Boot application (Java 21) that connects to the same `coe_dept` PostgreSQL database through Spring's `JdbcTemplate`, including the `finance.get_outstanding_fees()` function. Every functionality is a real operation — the service both **reads and writes** the database, and the Project 1 front end consumes it.

```
Browser  →  Next.js 14 front end (:3000)  →  Spring Boot API (:8080)  →  PostgreSQL coe_dept
```



The API landing page at `http://localhost:8080/`.

2. Technology & Structure

Concern	Choice
Language / runtime	Java 21
Framework	Spring Boot 3.5 (Spring Web)
Database access	Spring JDBC (`JdbcTemplate`) — runs the Project 1 SQL directly
Password hashing	BCrypt (`spring-security-crypto`)

Authentication	Stateless JWT (HS256), verified by a servlet filter
Build	Maven

The code is layered `controller → service → repository` (JdbcTemplate), with `dto`, `security` and `config` packages.

3. API Endpoints

Actions for the signed-in student live under `/api/me/*` (the student id is taken from the JWT). Endpoints marked "write" change the database.

Method & path	Type	Functionality
POST /api/auth/register	public	Create account + JWT
POST /api/auth/login	public	Login + JWT
GET /api/me	read	Profile + fee summary
PUT /api/me/profile	**write**	1 — edit contact details
POST /api/me/payments	**write**	2 — make a fee payment
GET /api/me/registrations	read	3 — current-term courses registered
GET /api/me/available-courses	read	3 — courses open to register
POST /api/me/enrollments	**write**	3 — register for a course
DELETE /api/me/enrollments/{id}	**write**	3 — drop a course
GET /api/me/grades	read	Completed results
GET /api/assignments/lecturer-course	read	4 — lecturer → course
GET /api/assignments/lecturer-ta	read	5 — lecturer → TA
GET /api/fees/outstanding	read	Project 1 JSON function (all students)

4. Authentication & Security

- **Passwords** are hashed with BCrypt; seeded and newly-registered accounts verify identically.
- **Login/registration** return a signed **JWT** (HS256) carrying the student id, email and expiry.
- A servlet filter (`JwtAuthFilter`) protects every `/api/**` route except `/api/auth/**`; a missing/invalid token yields `401` .
- **Ownership:** `/api/me/*` always acts on the token's own student, and the per-id read endpoints reject access to another student's records with `403` .
- **Validation:** payments must be positive and not exceed the outstanding balance (`400`); a duplicate course registration returns `409` .

5. Implementing the Project 1 Function

`GET /api/fees/outstanding` executes the Project 1 database function and streams its JSON straight to the client:

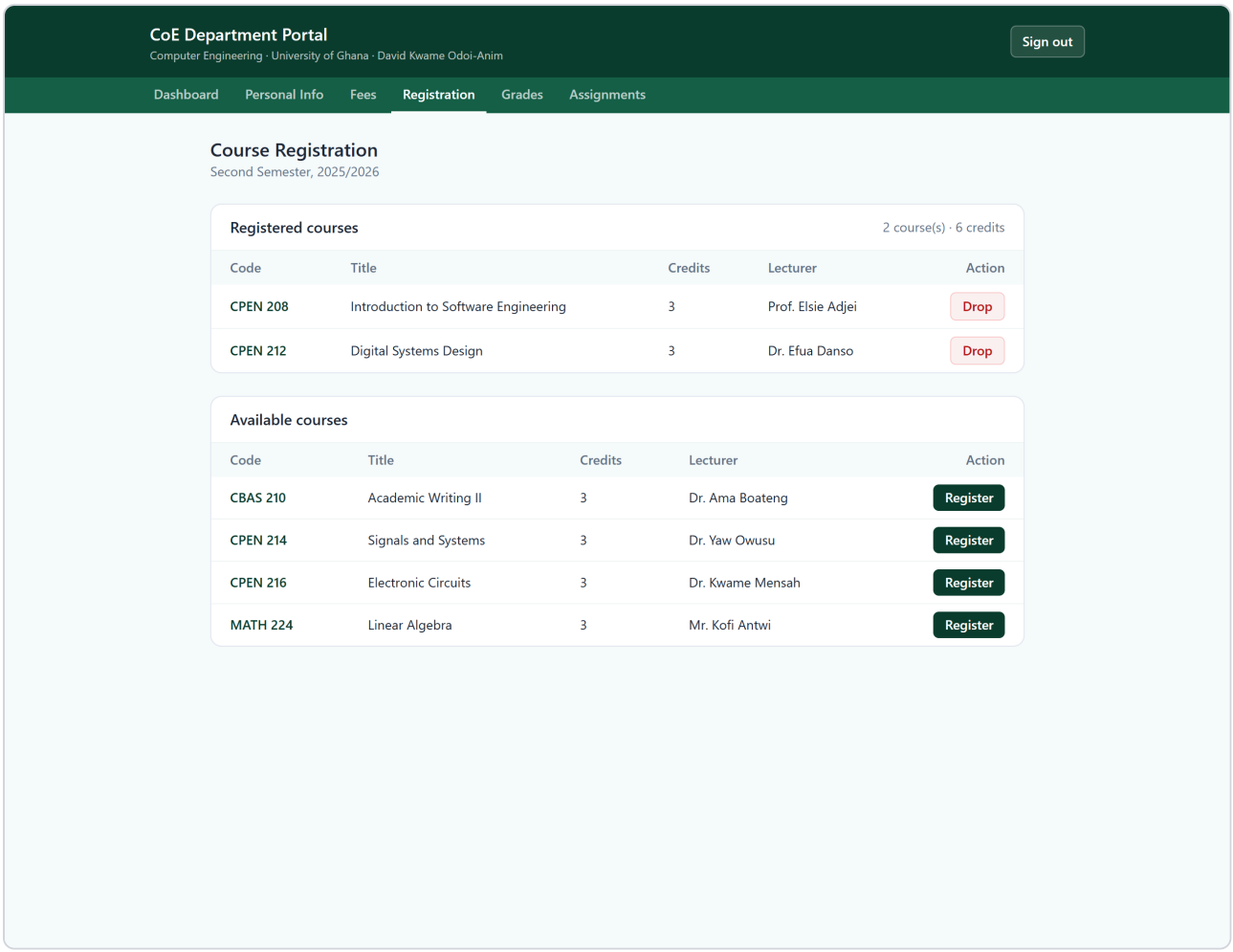
```
// FeesRepository.java
public String getAllOutstandingJson() {
    return jdbc.queryForObject(
        "SELECT finance.get_outstanding_fees()::text", String.class);
}
```

Write operations follow the same thin-repository style, e.g. recording a payment:

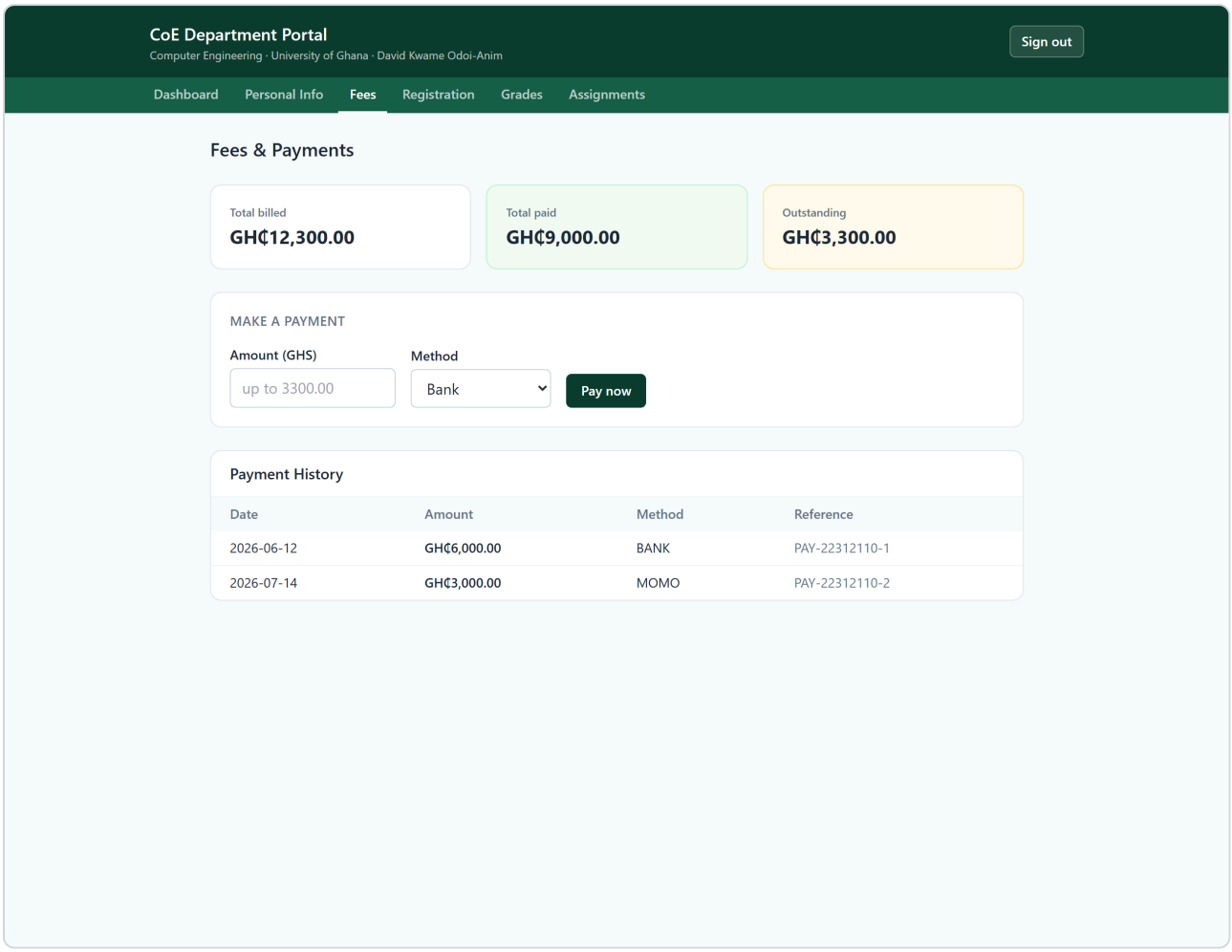
```
// FeesRepository.java
public PaymentDto insertPayment(long studentId, BigDecimal amount, String method) {
    return jdbc.queryForObject(
        "INSERT INTO finance.payments (student_id, amount, method, reference) " +
        "VALUES (?, ?, ?, 'PAY-' || ? || '-' || to_char(now(),'YYYYMMDDHH24MISS')) " +
        "RETURNING payment_id, amount, paid_on, method, reference",
        PAYMENT_MAPPER, studentId, amount, method, studentId);
}
```

5.1 The front end runs on this API

Every action on the Project 1 pages is served here. For example, the registration screen below calls `GET /api/me/registrations` and `/available-courses`, and its buttons call `POST` / `DELETE` `/api/me/enrollments`.



Course registration — Register/Drop buttons write to the database through this service.



Fees — the payment form posts to `/api/me/payments`; the balance then updates.

6. How to Run

1. Ensure the `coe_dept` database exists (Project 1 scripts or the provided backup).
2. `cd "PROJECT 2/api"`, set `DB_PASSWORD`, then `mvn spring-boot:run` (starts on `http://localhost:8080`).
3. Verify: `POST /api/auth/login` then, with the returned token, `GET /api/fees/outstanding` or any `/api/me/*` action.

Deliverables in this folder: Spring Boot source in `PROJECT 2/api`; database scripts and backup in `PROJECT 2/database`; full source on GitHub at `https://github.com/doxaodoi/project-1-2`.